Infosys Internship Learning Summary

Day 1 (10/11/25)

- **Introduction:** Overview of internship workflow, expectations, and guidelines.
- **Project Allocation:** Assigned the Smart Stock Inventory Optimisation project and responsibilities.

Day 2 (11/11/25)

- **Tech Stack Selection:** Decided on tools like Python, Fast API/Flask, Django, ML. model, and databases.
- **API Basics:** Understood how APIs allow communication between client and server.
- **Postman:** Learned to test and debug APIs using requests and responses.
- **WebSocket:** Understood real-time data exchange through persistent server connection.
- **NumPy:** NumPy is a Python library used for fast numerical operations on large arrays and matrices.
- **Pandas:** Pandas is a Python library that helps you handle, and analyse data using DataFrames.

Day 3 (12/11/25)

- **Fast API:** High-performance Python framework for building fast, modern API.
- **Flask API:** Lightweight Python framework used for simple and flexible API development
- **Pydantic**: Library for validating and converting data using Python models.

Day 4 (13/11/25)

- **GitHub**: Platform for code storage, version control, and team collaboration.
- **git init** ○ Creates a new Git repository in your project folder.

- **git clone <url>** ○ Copies an existing remote repository to your system.
- **git status** ○ Shows changes in your working directory.
- **git add.**
  ○ Stages all changed files for the next commit.
- **git add <filename>** ○ Stages a specific file.
- **git commit -m "message"** ○ Saves your staged changes with a message.
- **git push origin main** ○ Uploads your commits to the main branch on GitHub.
- **git pull** ○ Downloads and merges the latest changes from the remote repository.
- **git branch** ○ Shows all branches in your repository.
- **git branch <name>** ○ Creates a new branch.
- **git checkout <branch>** ○ Switches to another branch.
- **git merge <branch>** ○ Combines another branch into the current branch.
- **git log** ○ Shows commit history.
- **git remote -v** ○ Shows linked remote repositories.
- **Virtual Environment:** Isolated Python environment to manage project-specific packages.
- **requirements.txt:** File listing all dependencies needed to run the project.

Day 5 (14/11/25)

- **Payload:** The actual data sent in an API request or response.
- **Endpoint:** A specific API URL that performs a defined action.
- **Al Models**: Algorithms that learn patterns from data to make predictions or decisions.
- **Types of Machine Learning:** Supervised, unsupervised and reinforcement learning approaches.
- Llama Model: A large language model useful for NLP and text-generation tasks

Day 6 (16/11/25)

- **Database Concepts**: Learned tables, keys, relationships, and data storage principles.
- **Normalisation:** Process of organising data to reduce redundancy and improve consistency.
- **1NF-First Normal Form**: Data is organised so each column has atomic values and no repeating groups.
- **2NF-Second Normal Form:** The table is in 1NF and every non-key column depends on the primary key.
- **3NF Third Normal Form**: Table is in 2NF and has no transitive dependencies (non-key fields don't depend on other non-key fields).
- **BCNF - Boyce Codd Normal Form**: Stronger version of 3NF where every determinant must be a candidate key.
- **4NF-Fourth Normal Form:** Table must not contain a multivalued dependency.
- **5NF-Fifth Normal Form**: Data is broken into tables to avoid join dependency issues.
- **PostgreSQL vs MySQL:** PostgreSQL is feature-rich and strict, and MySQL is simpler and faster for basic use.
- **Vector Databases:** Databases that store embeddings for Al search, similarity, and recommendations.

Day 7 (17/11/25)

- **GitHub Repository:** A centralised place to store and manage version-controlled project code.
- **Project Review:** Evaluated current project progress and clarified next development steps.

Day 8 (18/11/25)

- Documents were reviewed and corrected.

Day 9(19/11/25)

- PPT presentation.
- Code was reviewed.

Day 10(20/11/2025)

- PPT presentation.
- Code was reviewed.

Day 11(21/11/25)

- Discussed about project.
- Reviewed the milestone.

Day 12(24/11/25)

Machine Learning Workflow:

- Collect Data: Gather all required raw data from various sources.
- Prepare and Engineer Features: Clean the data, handle noise, and create meaningful features for the model.
- Train a Model: Use machine learning algorithms to learn patterns from the prepared data.
- Evaluate on Holdout Data Using Metrics: Test the model on unseen data and measure performance using evaluation metrics.
- Deploy to Production and Monitor: Put the trained model into real use and continuously track its performance.

Applications Discussed:

- Demand Forecasting: Predict future product demand using machine learning methods.
- Dynamic Pricing: Adjust prices automatically based on demand, supply, and competition.
- Customer Analytics: Analyze customer behavior, segmentation, and patterns for business insights.

Day 13(25/11/25)

NLP points:

- **Sentiment Analysis**
  Understanding positive, negative, or neutral emotions from text.
  Used for reviews, feedback, and opinion mining.
- **Tokenization**
  Breaking text into smaller units like words or sub words.
  First step in most NLP pipelines.
- **Stemming**
  Reducing words to their root form.
  Example: *playing → play*
- **Lemmatization**
  Converting words to their dictionary/base form.
  Examples: *better → good*, *running → run*

- **Stop-Word Removal**
  Removing common words (the, is, in) that add little meaning.
  Helps reduce noise.
- **Text Preprocessing**
  Cleaning text before model building.
  Includes lowercasing, removing symbols, punctuation, numbers, etc.
- **Part-of-Speech (POS) Tagging**
  Identifying grammar roles such as noun, verb, adjective, etc.
  Helps understand sentence structure.
- **Named Entity Recognition**
  Identifying key entities in text.
  Types include PERSON, ORGANIZATION, GPE, DATE, PRODUCT
  Examples: dmart, India
- **Building small NLP models**
  General steps: preprocessing → feature engineering → training → evaluation

LSTM (Long Short-Term Memory):

A type of RNN used for sequential and time-dependent data.
Remembers long-term patterns and avoids forgetting.
LSTM Gates: Forget Gate, Input Gate, Output Gate.

Day 14(26/11/25)

1. **Margin:** Margin is the distance between the decision boundary and the nearest data points. A larger margin generally means better separation between classes.
2. **Hard margin vs Soft margin:**
   Hard margin allows *no misclassification* & fits data with perfect separating boundary. Data clear clean. There should not be noise.
   Soft margin allows some errors to avoid overfitting and handle noisy or overlapping data.
3. **TF-IDF:**
   a. TF (Term frequency) measures how often a word appears in a document. Higher TF means the word is more common within that specific document.
   b. IDF (Inverse Document Frequency) measures how rare a word is across all documents in a collection. Words that appear in fewer documents get higher IDF because they are more unique.
   c. Ex: “Git” word appears 30 times in the document and the total word in the document is 1000. “The” word appears 900 times in the document and the total word in the document is 1000.
   d. 30/1000 => High IDF and 900/1000 => low IDF
4. **Tools:**
   a. Calculator: used for computing TF, IDF values, ratios, margins, and numerical calculations.

b. Unit Converter: used for converting values during preprocessing or normalization steps.

5. **SVM:** Support vector machine
    a. SVM is a machine learning algorithm that finds the best boundary that separates classes.
    b. Works best for small or medium-sized datasets.
    c. Not suitable for highly noisy features because it tries to create a clear margin.
    d. Performs high-dimensional separation, can handle large feature spaces.
    e. Can solve non-linear problems using kernel tricks (RBF, polynomial, etc.).
    f. Gives strong accuracy when data is clean and well-separated.
    g. Focuses on maximizing margin, which makes the model more robust.
6. **Embedding:** Embedding converts raw text into numerical vectors so that a machine learning model can understand it.
7. **Reinforcement learning:** Reinforcement learning uses a try-and-error method where an agent learns from mistakes. The agent receives rewards or penalties and gradually learns the best actions.

Day 15(27/11/25)

- **Git vs GitHub**: Git is a local version control tool used to track code changes. GitHub is an online platform to store Git repositories and collaborate with others.
- **Agentic AI workflow:** Agentic AI follows a loop of planning, acting, observing, and improving. It continues autonomously until the goal is completed.
- **Types of agents:**

  • Reactive Agent

  A reactive agent responds instantly to the environment without memory. It makes decisions based only on current inputs, not past experiences.

  • Goal-Driven Agent

  A goal-driven agent selects actions based on achieving specific goals. It evaluates different actions and chooses the one that moves it closer to its goal.

  • Multi-Agent System

  A multi-agent system contains multiple agents that interact and collaborate. They work together to solve complex tasks that a single agent cannot handle efficiently. Manager agent will be there to look after the remaining agents works.
- **Autogen framework:** Autogen is a framework for building multi-agent AI systems. It lets different AI agents communicate, collaborate, and complete tasks automatically.
- **User proxy agent:** Mediator b/w user and assistant agent. A user proxy agent represents the user inside the agent system. It gives instructions, checks outputs, and ensures results match the user's needs.

- **Assistant agent:** An assistant agent is the main problem-solving agent. It performs tasks, generates outputs, and follows the instructions or plans provided.

Day 16 (28/11/2025)

1.OpenAI

- pip install openai
- resp = client. chat. completions. create
  (
  model= "gpt-4o-mini" #can replace with an available model like "gpt-4o" or "gpt-5" etc.
  messages = [
  {“role": "system", "content": "you are a helpful assistant”}
  {“role": "user", "content": "write a two-line poem about chair”}
  ]
  max_tokens = 120
)
- Response contains: model, messages, max_tokens

2. Code Structure (General Flow)

```
from openai import OpenAI
client = OpenAI()
assistant_text = resp.choices[0].message["content"]
```

3. Google Generative AI (Gemini)

- Installation: pip install google-generativeai
- Import: import google.generativeai as genai
- Model: model = genai.GenerativeModel("gemini-1.5-flash")
- User sends a message: response = model.generate_content("Write a short poem about coffee.")
- Chat Completions: model returns the next message.

4. Function Calling

```
functions = {
"name": "get_temperature"
"description": "returns temperature of a city"
"parameters": {
        "type": "object"
        "properties": {"city": {“type": "string"}}
            }}
```

Using function:

- model = genai.GenerativeModel("gemini-1.5-flash", tools=[functions])
- stream = True

5. **Flow**
   - Text → Tokens
     Input text is broken into tokens.
   - Tokens → Embeddings
     Tokens are converted into dense vectors.
   - Embeddings → Transformer Layers
     Transformer processes embeddings.
   - Attention Mechanism
     Model identifies which words are important.
   - Output → Text (decoded)
     Final generated/decoded text is returned.
6. **Temperature**
   - Controls creativity of output.
   - Range: **0.0 to 1.0**
7. **Rate Limits & Quotas**
   - Each API has usage limits depending on model and plan.
   - Limits apply to:
     - Requests per minute
     - Tokens per minute
     - Daily quotas